

AI Architect

CompTIA SecAI+: AI Security Certification Complete Exam Prep 2026 | Section 23: AI

1. The AI Architect's Domain: Systems, Not Models

If the Data Scientist works at the level of individual models, the AI Architect works at the level of entire systems. AI Architects make the structural decisions that determine whether a collection of models, pipelines, APIs, and data stores behaves as a coherent, secure, and scalable system. A brilliant model deployed in a poorly architected system will fail. An average model embedded in a well-designed architecture will outperform it. This gap - between model quality and system quality - is the AI Architect's domain.

For SecAI+ purposes, the AI Architect's significance lies in the security implications of architectural decisions. Choices about where data moves, how components communicate, which services manage credentials, and where trust boundaries fall are architectural decisions. When security gaps are traced back to structural design rather than individual component misconfiguration, they typically land in the AI Architect's accountability zone under NIST CSF 2.0 GV.RR.

2. NIST CSF 2.0 Accountability Mapping for AI Architects

NIST CSF 2.0 Function/Category	AI Architect Accountability
PR.PS - Platform Security	Ensure platforms hosting AI workloads are configured securely: network segmentation, access controls, secure defaults across training and serving infrastructure
GV.RM - Risk Management	Evaluate and document risk implications of major technology choices: cloud vs. on-premise, open-source vs. proprietary, sync vs. async inference
PR.AA - Identity Management	Design authentication and authorization across all pipeline components, including service-to-service access within the ML pipeline
DE.CM - Monitoring	Design observability infrastructure that surfaces security events from all pipeline components, not just application-layer metrics
ID.AM - Asset Management	Maintain architectural diagrams and component inventories as direct inputs to threat modeling exercises

Architectural decisions made without formal risk documentation create governance gaps that compliance audits will flag. ISO/IEC 42001:2023 specifically requires documentation of AI system design decisions as part of AI management system requirements. The AI Architect is the practitioner whose role makes this documentation tractable - they hold the end-to-end system view that no other role has.

3. End-to-End System Thinking: The Architect's Distinctive Competency

The distinctive competency of an AI Architect is end-to-end thinking. Most practitioners - Data Scientists, ML Engineers, Data Engineers - focus on their segment of the pipeline. The AI Architect must hold the

entire system in view simultaneously: how data enters, how it is stored, how models are trained and validated, how predictions reach end users, how the system degrades under load, and how it behaves when individual components fail.

This systems perspective prevents a class of problems that appear only when the pieces are assembled: integration failures, emergent performance bottlenecks, security gaps at component boundaries, and monitoring blind spots. Architects who think only about the happy path create systems that surprise operators at exactly the wrong moment. For SecAI+ exam scenarios, end-to-end thinking is the lens through which architectural accountability questions are answered.

4. Technology Selection and Its Long-Term Security Consequences

AI Architects make technology choices that organizations live with for years. Each choice carries security implications that compound over time:

- Open-source frameworks with large communities receive faster vulnerability patches - commercial platforms offer vendor support but create dependency.
- Homegrown solutions give maximum control but concentrate expertise dangerously in a small team - creating key-person risk and audit difficulty.
- Orchestration platform choice determines where pipeline audit logs live and who can access them.
- Feature store selection determines centralization of access control for training data access paths.
- Inference serving framework choice determines default authentication behavior - most serve unauthenticated by default.

AI Architects who treat these as purely technical decisions miss the security and organizational risk dimensions that compound over time. The EU AI Act (2024) and NIST AI RMF 1.0 both require organizations to document the rationale for major design choices in high-risk AI systems - documentation that falls squarely on the AI Architect to produce and maintain.

5. Defense in Depth Across the AI Stack

AI Architects implement defense in depth across all layers of the AI system. Authentication and authorization must cover not just user access to AI interfaces but service-to-service access within the pipeline - model training jobs calling data stores, inference services querying feature stores, monitoring systems reading prediction logs. Each service-to-service interaction is an authorization boundary that must be explicitly designed and audited.

Data protection requires encryption at rest and in transit across all components, with key management handled through dedicated services (AWS KMS, Azure Key Vault, HashiCorp Vault) rather than hardcoded credentials. Network segmentation isolates training infrastructure from inference infrastructure from monitoring infrastructure - a breach in one environment should not provide lateral movement to others. The principle of least privilege applies as strictly to ML training jobs as to human users.

Security Design Principle: AI systems are not exempt from standard security controls. Architects who treat AI components as special cases requiring different security standards create systems that will eventually violate their organization's own security policies and fail compliance audits.

6. Data Architecture: The Foundation Everything Else Rests On

AI systems are only as reliable as their data infrastructure, and AI Architects design that infrastructure. Critical decisions include: where raw data lives and how it is partitioned, how transformation pipelines convert raw data to model-ready features, how feature stores serve consistent features across training and inference (preventing training-serving skew), and how data governance controls - lineage tracking, access auditing, retention policies - are implemented.

Feature stores deserve special emphasis: they centralize access control for feature data. Instead of each training pipeline independently accessing raw data sources (creating multiple access paths, each a potential vulnerability), the feature store enforces who can read which features from a single controlled interface. This centralization is simultaneously an engineering benefit and a security benefit. Architects who design data infrastructure well make every downstream practitioner's security posture stronger.

7. Real-World Incident: Misconfigured AI Pipeline Exposes PII

Real-World Incident - AI Pipeline Storage Misconfiguration (2023): A security research team documented a case where an AI model training pipeline stored intermediate features in an S3 bucket with overly permissive ACLs - a misconfiguration left by a developer during initial setup and never corrected. An attacker with network access discovered the bucket through automated scanning, downloaded the cached features, and reverse-engineered the training dataset structure. The features contained PII. The root cause was architectural: no process existed to audit storage permissions after initial deployment.

The defense is architectural: security review gates built into the architecture definition process, not left as assumptions for downstream teams. AI Architects who design pipelines with automated permission auditing - checking storage bucket ACLs, service account permissions, and network access controls as part of deployment pipelines - prevent this class of vulnerability. The misconfiguration was visible and correctable; the gap was process design, an architectural responsibility.

8. Real-Time vs. Batch Architecture: Security Tradeoffs

Architecture Type	Latency	Complexity	Security Considerations
Real-time (streaming)	Milliseconds	High - streaming failure modes	Ephemeral data complicates audit logging; harder to replay for forensic analysis
Batch processing	Minutes to hours	Lower - scheduled jobs	Creates persistent intermediate artifacts requiring access controls
Hybrid (both)	Both as needed	Highest - two integration surfaces	Two distinct security models must be maintained; integration layer is the gap most often exploited

SecAI+ exam scenarios may describe a requirement and ask which architecture is appropriate. Real-time systems process data ephemerally - this complicates audit logging because there is no persistent intermediate artifact to inspect. Batch systems create persistent intermediate artifacts - these require

explicit access controls. The AI Architect who chooses the processing model implicitly chooses the associated security model and must design accordingly.

9. Multi-Model Orchestration and Its Security Implications

Modern AI systems often involve multiple models working in sequence or parallel. A fraud detection system might use one model to flag suspicious transactions, a second to assess account risk, and a third to generate explanations for human reviewers. The AI Architect designing this system must consider: how one model's output feeds the next, what happens when one model fails or produces low-confidence outputs, how versions of each model are managed independently, and how the combined system's output is audited when tracing back through multiple models.

MITRE ATLAS v2.1 documents attacks that exploit inter-model dependencies - where manipulating one model's inputs produces predictable errors in downstream models. This cascading failure mode does not exist in single-model systems but is a direct consequence of multi-model orchestration. AI Architects who model these dependencies during design can implement circuit breakers, output validation between models, and confidence thresholds that prevent cascading failures from amplifying into system-level security events.

10. Red Team Perspective: Mapping the AI Attack Surface

A red team analyzing an AI system's architecture looks for: data ingestion endpoints that accept external input without validation, model serving APIs that do not rate-limit or authenticate callers, feature stores with over-permissioned service accounts, training pipelines that pull dependencies from public repositories without integrity checks, and monitoring systems that could be manipulated to suppress alerts.

The architectural pattern red teams find most exploitable is component sprawl - AI systems assembled from many loosely integrated services where security was never considered at the integration layer. AI Architects who think like attackers during design identify and close these gaps before they are discovered in production. Conducting formal threat modeling (using STRIDE or MITRE ATT&CK for Cloud) against the architecture diagram is a practice that converts red team perspective into proactive security design.

11. Documentation as a Security Artifact

Architecture documentation serves security purposes beyond onboarding new engineers. Architecture diagrams showing data flows, trust boundaries, and component interactions are direct inputs to threat modeling. Decision records explaining why specific technology choices were made help future architects understand constraints and avoid well-intentioned changes that reintroduce known risks. Reference architectures encoding security patterns ensure new projects start from a secure baseline.

Organizations that treat architecture documentation as optional find their threat models are perpetually outdated because nobody documented the system as it evolved. ISO/IEC 42001:2023 specifically requires documentation of AI system design decisions. The AI Architect who maintains living architecture

documentation is creating a security artifact with lasting value - one that compliance auditors, incident responders, and successor architects will rely on.

Key Terms Glossary

Term	Definition
End-to-end system thinking	The AI Architect's distinctive competency: holding the entire pipeline in view simultaneously to identify integration failures, security gaps at component boundaries, and monitoring blind spots.
Defense in depth	Security principle applied to AI systems: multiple overlapping controls across authentication, encryption, network segmentation, and least privilege, so that failure of any single control does not result in a breach.
Feature store	Centralized system that computes and serves features consistently across training and inference, both preventing training-serving skew and centralizing access control for feature data.
Training-serving skew	Divergence between how features are computed during training versus inference; an architectural failure mode that degrades model performance in production.
Component sprawl	AI systems assembled from many loosely integrated services where security was not considered at integration points; the architectural pattern most exploited by red teams.
PR.PS (NIST CSF 2.0)	Platform Security - the subcategory requiring secure configuration of platforms hosting AI workloads, directly within the AI Architect's accountability zone.
GV.RM (NIST CSF 2.0)	Risk Management governance subcategory requiring formal documentation of technology choices and their risk implications; drives AI Architect's decision records.
Multi-model orchestration	Architecture pattern where multiple models work in sequence or parallel; introduces cascading failure modes and inter-model dependency attack surfaces cataloged in MITRE ATLAS v2.1.

Quick Revision Checklist

- AI Architect works at system level - models, pipelines, APIs, and data stores as a coherent whole.
- NIST CSF 2.0 PR.PS and GV.RM are the primary accountability categories for AI Architects.
- End-to-end thinking prevents integration failures and security gaps at component boundaries.
- Technology selection has long-term security consequences - open-source, commercial, and homegrown choices each carry distinct risk profiles.
- Defense in depth applies to AI systems: authentication, encryption, network segmentation, and least privilege across all components.
- Feature stores centralize access control for feature data - this is both an engineering and a security benefit.
- S3/storage bucket ACL misconfiguration is a documented, high-frequency AI pipeline vulnerability - design automated permission auditing into deployment pipelines.

- Multi-model orchestration creates cascading failure modes not present in single-model systems.
- Architecture diagrams are threat modeling inputs - living documentation is a security artifact.
- ISO/IEC 42001:2023 requires documentation of AI system design decisions - AI Architect is the practitioner who can produce this.

10 Exam-Based MCQs - AI Architect Role and System Security

Test yourself after reading. These questions are written in real exam style - scenario-heavy, with plausible distractors. Read every explanation, including for questions you answered correctly.

Q1. Scenario: A red team exercise at a healthcare AI company discovers that an S3 bucket containing cached training features is publicly accessible. The bucket was created 18 months ago during initial pipeline setup. No one changed the permissions - they were simply never reviewed after the initial developer set them as open for testing convenience. During a post-incident review, investigators trace a PII exposure to an S3 bucket with overly permissive ACLs used by an ML training pipeline. No process existed to audit storage permissions after initial deployment. Which role is PRIMARILY accountable for this gap?

- A) Data Scientist, because they chose to use S3 for training data storage
- B) ML Engineer, because they configured the training pipeline that wrote to the bucket
- C) AI Architect, because the absence of a permission auditing process is an architectural gap
- D) Cloud Security Engineer, because S3 security is a cloud infrastructure responsibility

Answer: C **Explanation:** C is correct because the missing process - automated permission auditing after deployment - is an architectural design gap. The AI Architect is responsible for designing security review gates into the pipeline architecture, not leaving them as assumptions for individual practitioners to handle independently. A is wrong because data storage choice is an input to architectural design, not a root cause for missing process governance. B is wrong because the ML Engineer followed the architectural design; the gap was that no auditing process was designed. D is wrong because cloud security engineers configure infrastructure per architectural specifications; if the specification didn't include permission auditing, the gap is in the architecture.

Q2. An AI Architect is designing a multi-model fraud detection system where a transaction classifier feeds output to a risk scorer, which feeds to an explanation generator. Which security concern is UNIQUE to this multi-model architecture?

- A) The serving APIs must use HTTPS to encrypt traffic between models
- B) Adversarial manipulation of the transaction classifier could produce cascading errors in downstream models
- C) The explanation generator must not log PII in its outputs
- D) Model artifacts must be stored in a version-controlled registry

Answer: B **Explanation:** B is correct because cascading failure through inter-model dependencies is a security risk that does not exist in single-model systems. MITRE ATLAS v2.1 documents attacks that exploit these dependencies - manipulating one model's inputs to produce predictable errors in downstream models. A is wrong because HTTPS encryption is a universal best practice that applies equally to single-

model and multi-model systems. C is wrong because PII handling in logs is a data protection concern that applies to any model, not specific to multi-model architecture. D is wrong because model versioning applies to all deployment scenarios, not specifically to multi-model orchestration.

Q3. An organization is choosing between batch processing and real-time streaming for an AI-based threat detection system. From a security audit perspective, which is the PRIMARY advantage of batch processing?

- A) Batch systems have lower latency, enabling faster threat response
- B) Batch systems create persistent intermediate artifacts that can be inspected for forensic analysis
- C) Batch systems require fewer access controls because data is processed less frequently
- D) Batch systems eliminate training-serving skew by processing all data identically

Answer: B **Explanation:** B is correct because batch systems create persistent intermediate artifacts - processed data files, feature datasets, prediction logs - that are available for forensic inspection after an incident. Real-time streaming processes data ephemerally, making post-incident audit logging more complex. A is wrong because batch systems have higher latency than real-time systems - this is a disadvantage for threat response speed, not an advantage. C is wrong because batch systems still require access controls on their persistent artifacts; less frequent processing does not reduce the security requirement. D is wrong because training-serving skew is addressed by feature stores and consistent preprocessing logic, not by processing frequency.

Q4. Under NIST CSF 2.0, which subcategory MOST directly holds the AI Architect accountable for documenting the risk implications of choosing an open-source ML framework over a commercial platform?

- A) PR.PS - Platform Security
- B) GV.RM - Risk Management
- C) DE.CM - Continuous Monitoring
- D) RS.AN - Incident Analysis

Answer: B **Explanation:** B is correct because GV.RM (Risk Management) requires formal evaluation and documentation of risk implications for major organizational and technology choices. The decision to use open-source versus commercial frameworks carries risk implications (patch velocity, support availability, vulnerability management) that GV.RM requires to be documented. A is wrong because PR.PS governs secure platform configuration, not the risk evaluation of technology selection decisions. C is wrong because DE.CM governs ongoing monitoring of deployed systems, not design-time technology choices. D is wrong because RS.AN governs incident analysis after events occur, not proactive risk documentation.

Q5. A red team discovers that a model serving API accepts inference requests without authentication. The AI Architect argues that network perimeter controls prevent unauthorized external access. Which BEST describes the risk of this architectural assumption?

- A) The assumption is sound if the network perimeter uses next-generation firewall technology

- B) Network perimeter controls alone are insufficient; any internal network compromise provides unrestricted model access
- C) Unauthenticated APIs are acceptable for internal-only ML systems with low-sensitivity outputs
- D) The risk is negligible because model extraction attacks require thousands of queries, which would be detected

Answer: B **Explanation:** B is correct because perimeter-only security violates defense-in-depth principles. If any internal system is compromised - through phishing, lateral movement, or insider threat - the unauthenticated model API is immediately accessible. Model extraction attacks, unauthorized query logging, and denial-of-service via inference flooding all become possible. A is wrong because NGFW technology does not compensate for missing application-layer authentication; it addresses perimeter threats, not internal access. C is wrong because 'internal-only' is not a stable security property - internal networks are frequently compromised. D is wrong because detection-based arguments do not prevent the attack; they assume detection will work, which is not guaranteed.

Q6. Which architectural pattern BEST prevents training-serving skew in a production AI system?

- A) Deploying the model training code and inference code in the same container
- B) Using a centralized feature store that computes and serves features identically during training and inference
- C) Retraining the model weekly to keep it aligned with production data distributions
- D) Implementing A/B testing to compare training and production model behavior

Answer: B **Explanation:** B is correct because training-serving skew arises when features are computed differently during training versus inference. A centralized feature store that serves the same computed features to both training pipelines and inference services eliminates the possibility of divergence. A is wrong because co-locating code in the same container does not guarantee the code computes features identically in both contexts. C is wrong because retraining addresses model staleness from concept drift, not the feature computation inconsistency that causes training-serving skew. D is wrong because A/B testing compares model version performance, not feature computation consistency.

Q7. ISO/IEC 42001:2023 requires documentation of AI system design decisions as part of an AI management system. Which AI role is PRIMARILY responsible for creating this documentation?

- A) Data Scientist, because they make the modeling decisions
- B) ML Engineer, because they implement the final production system
- C) AI Architect, because they hold the end-to-end system design view
- D) AI Governance Engineer, because governance documentation is a governance function

Answer: C **Explanation:** C is correct because ISO/IEC 42001:2023 design documentation requires a view of the entire system - data flows, component choices, trust boundaries, and technology selection rationale. Only the AI Architect holds this end-to-end view. A is wrong because Data Scientists document model-level decisions (model cards), not system-level architecture. B is wrong because ML Engineers document implementation details of specific pipelines, not system architecture. D is wrong because AI Governance Engineers maintain governance processes and audit trails, but rely on architects to produce the underlying

design documentation.

Q8. An AI Architect proposes an autonomous loan denial system with no human review step, arguing that the model's 96% accuracy makes human oversight unnecessary. Which regulatory requirement does this architectural decision MOST directly violate?

- A) NIST CSF 2.0 DE.CM - the system lacks continuous monitoring
- B) EU AI Act (2024) - high-risk AI applications require human oversight mechanisms in the architecture
- C) ISO/IEC 42001:2023 - AI management systems must document all model training decisions
- D) GDPR - automated processing of personal data requires explicit consent

Answer: B **Explanation:** B is correct because the EU AI Act (2024) explicitly requires that high-risk AI systems - which include AI used in credit decisions - incorporate human oversight mechanisms. The absence of a human review step is a direct architectural violation of this requirement, regardless of model accuracy. A is wrong because DE.CM governs monitoring of deployed systems; the issue here is human oversight in the decision process, not monitoring. C is wrong because ISO/IEC 42001:2023 model training documentation is a relevant requirement but not the one most directly violated by removing human review. D is wrong because GDPR's automated processing provisions (Article 22) require the right to human review but the most direct regulatory framework for AI oversight requirements is the EU AI Act.

Q9. Which of the following BEST describes the security risk introduced by architectural component sprawl in AI systems?

- A) Sprawling architectures require more compute resources, increasing cloud costs
- B) Security was not considered at integration points between loosely coupled components, creating exploitable gaps
- C) Component sprawl makes model training slower, reducing the frequency of security updates
- D) Sprawling architectures are harder to document, increasing compliance audit burden

Answer: B **Explanation:** B is correct because component sprawl - the pattern most exploited by red teams per MITRE ATT&CK and practical AI security assessments - means security was never designed at the integration layer between components. Each integration point is a potential attack surface: unauthenticated internal APIs, unencrypted inter-service traffic, over-permissioned service accounts. A is wrong because compute cost is a performance concern, not a security risk. C is wrong because training speed is unrelated to the integration-layer security gaps introduced by sprawl. D is wrong because audit burden is a compliance concern, not the primary security risk; the exploitation of integration-layer gaps is the direct security consequence.

Q10. A SecAI+ scenario describes an AI system that behaved correctly during testing but failed in a way that traced back to a gap at the interface between the feature store and the model serving component. Which practitioner is MOST accountable for this failure?

- A) Data Scientist, because they designed the features used by the model
- B) ML Engineer, because they built the serving pipeline that consumes features

C) AI Architect, because component interface design and integration security are architectural responsibilities

D) Data Engineer, because the feature store is a data infrastructure component

Answer: C **Explanation:** C is correct because gaps at component interfaces are architectural failures. The AI Architect is responsible for designing the integration between components - specifying what each interface must enforce (authentication, data validation, error handling) and ensuring that no security gap exists at component boundaries. A is wrong because the Data Scientist designs features, not the integration interface between the feature store and serving layer. B is wrong because the ML Engineer implements the serving pipeline within the architectural specification; if the specification did not require interface security, the gap is in the architecture. D is wrong because the Data Engineer builds data pipelines to architectural specifications; the interface design is the Architect's responsibility.